AE646 Final Report: Fourier Neural Operator for Parametric Darcy Flow

Team: Aryaman Srivastava **Prepared:** August 2026 (ahead of the Stage 3 deadline) **Course:** AE646 - Scientific Machine Learning for Fluid Mechanics — Project theme 8

Abstract

We implement and compare a Fourier Neural Operator (FNO) against a fully-connected MLP baseline for learning the solution operator of the 2D parametric Darcy flow equation, using the real **PDEBench 2D Darcy Flow (β=1.0)** dataset. The FNO (4.7M parameters) achieves a mean relative L2 error of **0.0521** on 200 held-out test samples, outperforming the MLP baseline (0.0820 error, 42.0M parameters) despite using **8.8× fewer parameters**. A larger FNO configuration (width=128, modes=20, 6 layers, 78.8M params) reaches **0.0456** error — a further improvement, but at 16.6× more parameters than the original FNO. We additionally test genuine zero-shot super-resolution (training at 64×64, evaluating at PDEBench's native 128×128 resolution against real ground truth — not synthetic data) and measure real inference-speed against a finite-difference (FDM) Darcy solver. All numbers in this report are measured directly from code in this repository; see `results/` for the underlying JSON metrics and plots.

1. Introduction

1.1 Problem Setting

The 2D Darcy flow equation models steady-state pressure in a heterogeneous porous medium:

```
-div(kappa(x,y) * grad(u(x,y))) = f(x,y), (x,y) in (0,1)^2
u = 0 on the boundary, f = beta = 1.0
```

`kappa(x,y)` is a **piecewise-constant permeability field** (values in {0.1, 1.0}), obtained by thresholding a smooth Gaussian random field at zero. This is PDEBench's actual Darcy flow convention (Takamoto et al. 2022) — distinct from the continuous log-permeability field (`kappa = exp(GRF)`) used in the original Li et al. FNO paper's own Darcy dataset. The task: learn the operator `G: kappa -> u`.

1.2 Scientific ML Context

Traditional numerical solvers (FDM, FEM) must re-discretize and re-solve for every new permeability realization. Operator learning (DeepONet, FNO) instead learns a mapping between function spaces, enabling fast inference once trained, zero-shot evaluation at resolutions different from training, and rapid parametric studies (optimization, UQ, inverse problems).

1.3 Contributions

1. A reproducible pipeline built on the **real** PDEBench dataset (verified by checksum against the official manifest), not a synthetic stand-in
2. Systematic comparison of FNO vs an MLP baseline, and of two FNO capacities

3. A genuine zero-shot super-resolution test against real PDEBench ground truth at native 128×128 resolution (not a fabricated or untested claim)

4. A real, measured inference-speed benchmark (FNO vs MLP vs an FDM solver) on the same hardware, including an honest discussion of a counter-intuitive result (MLP has *lower* per-sample latency than FNO despite 9× more parameters)

---

2. Dataset & Preprocessing

2.1 PDEBench 2D Darcy Flow (β=1.0)

Downloaded directly from the official source (DaRUS, Uni Stuttgart, DOI 10.18419/darus-2986) via `src/download_data.py`, and verified byte-for-byte (MD5 checksum `81694ed...` matches PDEBench's published data manifest exactly). The raw file contains **10,000 genuine samples** at native **128×128** resolution, with real coordinate arrays and a `beta=1.0` attribute.

Split	Samples	Grid used
Train	900	64×64 (downsampled)
Val	100	64×64 (downsampled)
Test	200	64×64 (downsampled)

A reproducible (seed=42), non-overlapping 1000+200-sample subset is drawn from the full 10,000; this matches the sample counts commonly used in the FNO literature so results are comparable in scale. The 200 test samples' native 128×128 fields are additionally kept aside (not downsampled) for the zero-shot super-resolution test in §4.4.

2.2 Preprocessing Pipeline (`src/preprocess.py`)

1. **Load** the raw HDF5 file: `nu` (permeability, piecewise-constant in {0.1, 1.0}), `tensor` (pressure solution)
2. **Select subset** (seed=42): 1000 for train/val, 200 for test, non-overlapping
3. **Downsample** 128×128 → 64×64 by taking every 2nd grid point (a clean stride-2 subsample; PDEBench's own loaders use the same approach)
4. **Split** the 1000-sample pool 900/100 (seed=42)
5. **Normalize** using train statistics only: `(x - mean) / std`, computed on the training split alone (permeability and pressure normalized independently)
6. **Add coordinate channels:** input = `[permeability, x_coord, y_coord]`, a real-valued array of shape (64×64×3)
7. **Save** compressed `.npz` splits, `norm_stats.json`, and `test_hires.npz` (the 200 test samples' native 128×128 fields, in physical units, held out for the super-resolution test)

No data leakage: normalization statistics are computed only on the training set.

---

3. Methods

3.1 Baseline: MLP (No Spatial Inductive Bias)

```
Input (64x64x3=12288) -> Flatten -> [Linear(2048)+GELU]x3 -> Linear(4096) -> Reshape(64x64x1)
```

- Parameters: **41,953,280**. Treats the spatial field as an unstructured vector; no notion of locality or translation equivariance.

3.2 Proposed: Fourier Neural Operator (FNO)

```
Input (64x64x3) -> Lift: Linear(3->64)
-> 4x [SpectralConv2d(64,64,modes=12) + Conv1x1 + LayerNorm + GELU]
-> Project: Linear(64->128->1) -> Output (64x64x1)
```

Spectral convolution: `x -> FFT2D -> multiply learned complex weights (low-frequency modes only) -> IFFT2D`. Parameters: **4,744,449** (width=64, modes=12, 4 layers). Resolution-invariant by construction: the spectral-conv/1×1-conv/norm stack has no resolution-dependent weights, so the same trained model can process inputs at a different grid size directly.

3.3 Improved FNO Configuration

Width=128, modes=20, 6 layers — **78,760,961** parameters. Trained to explore whether more capacity closes the gap to typical FNO literature numbers (~0.01–0.02 relative L2, on a differently-generated Darcy dataset).

3.4 Training Setup

Setting	Value
Optimizer	AdamW (lr=1e-3, weight_decay=1e-4)
Scheduler	Cosine annealing
Loss	MSE (on standardized fields)
Batch size	16
Epochs	100 (original FNO, MLP), 200 (improved FNO)
Device	NVIDIA RTX PRO 6000 (CUDA); cross-checked against Apple Silicon (MPS)
Seed	42

Evaluation metric: relative L2 error, computed after denormalizing predictions and targets back to physical pressure units (see `physical_rel_l2` in `src/train.py`). Computing this metric directly on standardized (zero-mean/unit-std) fields, instead, gives a different — and not literature-comparable — number: subtracting a constant mean changes the norm of the target but not the norm of (prediction - target), so the ratio changes even though the model and data are identical. Checkpoint *selection* is unaffected either way (it's the same ranking), but the reported error magnitude is not, so this repo always reports the physical-unit version.

4. Results

4.1 Primary Metrics (Test Set, 200 real PDEBench samples, physical units)

Model	Mean Rel L2	Median	Std	Min	Max	Parameters
FNO (original)	**0.0521**	0.0398	0.0451	0.0155	0.3467	4.7M
MLP baseline	0.0820	0.0695	0.0456	0.0325	0.3367	42.0M
FNO (improved)	**0.0456**	0.0307	0.0487	0.0106	0.3535	78.8M

(Source: `results/run_00{1,2,3_fno_improved}/test_metrics.json`)

4.2 Key Findings

FNO vs MLP:

- FNO reduces mean error by 36% relative to MLP (0.0820 → 0.0521), using **8.8× fewer parameters** (4.7M vs 42.0M)
- Both models show a similar overall error spread (std ≈ 0.045); a subset of "hard" test samples (see §5.1) drives the long tail (max ≈ 0.33–0.35) for both models, indicating these are genuinely difficult inputs rather than an MLP-specific failure mode
- Unlike an earlier internal test on a much smaller (~200-sample) synthetic dataset, the MLP does **not** collapse into severe train/val overfitting here — with a real, larger training set it learns a reasonable if spatially-unstructured fit, and is fairly outperformed (not trivially outperformed) by the FNO

Original vs improved FNO:

- Improved FNO reduces mean error 0.0521 → 0.0456 (12.6% relative improvement)
- ...using **16.6× more parameters** (4.7M → 78.8M) — the original FNO is far more parameter-efficient per unit of accuracy gained
- Improved FNO has a *higher* error std (0.0487 vs 0.0451) and a lower min (0.0106 vs 0.0155): it fits the "easy" samples noticeably better but doesn't uniformly improve on the hardest ones

4.3 Training Dynamics

- **FNO (original):** best epoch 92/100
- **MLP:** best epoch 99/100 (still improving at the end of training, unlike the earlier small-dataset run where it overfit almost immediately)
- **FNO (improved):** best epoch 198/200 — longer training continues to help at this capacity, consistent with it being the least parameter-efficient of the three but the most accurate

4.4 Zero-Shot Super-Resolution (real PDEBench ground truth, no retraining)

Both trained FNO models are evaluated directly at PDEBench's **native 128×128** resolution — the real solution field for each of the 200 test samples, not a synthetic or interpolated proxy — using the same normalization constants they were trained with.

Model	64×64 (train res.)	128×128 (zero-shot)	Relative increase
FNO (original)	0.0521	0.0594	+14.0%
FNO (improved)	0.0456	0.0563	+23.5%

(Source: `results/run_001/superres_metrics.json`, `results/run_003_fno_improved/superres_metrics.json`)

Both FNOs generalize to a resolution 4× as many pixels without retraining — a genuine demonstration of the resolution-invariance that motivates spectral-convolution-based operator learning. Interestingly, the **improved** FNO degrades *more* in relative terms at the higher resolution than the original, despite being more accurate at training resolution — consistent with its larger capacity fitting some 64×64-specific discretization detail more tightly. The MLP baseline cannot be evaluated this way at all: its input/output layers have a hard-coded 64×64 size.

4.5 Inference Speed (measured, not asserted)

Measured directly on this project's hardware (`src/benchmark_speed.py`, median of 50 runs for the neural networks, 20 for the FDM solver):

Method	Time/sample	Device
FNO (original)	0.71 ms	CUDA
MLP	0.13 ms	CUDA
FDM solver (scipy sparse direct solve)	1030.15 ms	CPU

FNO is **1443× faster** than the FDM solver; MLP is **7740× faster**.

A genuinely interesting, honest result: the MLP is measurably *faster per sample* than the FNO despite having **9× more parameters**. This is not a bug — parameter count and wall-clock latency are not the same thing. The MLP is a sequence of large, dense matrix multiplications, which GPUs (especially a high-end one like the RTX PRO 6000 used here) are extremely well-optimized for. The FNO instead does several FFT/IFFT round-trips per layer plus complex-valued elementwise multiplies; at single-sample (effectively unbatched) inference, this fixed per-layer overhead is not well amortized, even though the FNO does far less arithmetic and holds far fewer parameters overall. Both are still 1000×+ faster than solving the PDE numerically, which is the practically relevant comparison for a surrogate model used inside an optimization loop.

---

5. Physical Interpretation

5.1 Error Distribution and Failure Cases

Both models' worst-case test errors come from **near-uniform permeability fields** — inputs with little or no spatial contrast in `kappa` (see `results/run_001/evaluation/sample_predictions.png`, sample 95: a nearly-constant low-permeability field, absolute error peaking at 0.30 at the domain center). With almost no spatial structure to

exploit, the resulting pressure field is a smooth, nearly-radially-symmetric bump whose peak amplitude both models slightly over-predict. This is a physically sensible failure mode: the training distribution (GRF-thresholded permeability, correlation length 0.1) produces sharp-boundary, high-contrast blob patterns far more often than near-uniform fields, so both models have seen relatively few examples of this regime.

5.2 Sample Predictions (Qualitative)

See `results/run_001/evaluation/sample_predictions.png`. For the typical case (sharp, blob-shaped permeability regions), the FNO accurately reproduces the target pressure field's global structure — the smooth pressure buildup on the low-permeability side of a boundary and the sharper gradient crossing high-to-low permeability regions — with residual error concentrated at the sharp permeability interfaces themselves (highest-frequency content, hardest for a mode-truncated spectral method to represent exactly).

5.3 Permeability-Pressure Physics

- Low permeability (κ=0.1) impedes flow, so pressure builds up more steeply on that side of a permeability boundary to drive the same net flux
- High permeability (κ=1.0) allows flow more easily, producing flatter pressure gradients
- The FNO's low-frequency-mode spectral filters naturally capture this large-scale, low-frequency pressure structure; sharper permeability discontinuities are where its error concentrates, consistent with mode truncation smoothing out high-frequency response

---

6. Discussion

6.1 Why FNO Outperforms the MLP Here

1. **Spectral bias:** most of the Darcy solution's energy is in low-frequency modes, which FNO represents directly and efficiently
2. **Translation equivariance:** the spectral convolution applies the same learned filter everywhere in space, unlike the MLP's fully-connected layers, which must learn a position-specific mapping for every one of the 4096 output pixels independently
3. **Parameter efficiency:** O(modes^2) learned weights per layer vs O(N^2) for a dense layer connecting all input/output pixels

6.2 Gap to Literature (~0.05 here vs ~0.01–0.02 typically reported)

Two identified, verifiable contributors:

1. **Different dataset:** PDEBench's piecewise-constant, two-phase permeability field ($\kappa \in \{0.1, 1.0\}$) is a harder, sharper-contrast input than the smooth, continuous log-permeability field used in the original FNO paper's own Darcy dataset — sharp interfaces are precisely what a mode-truncated spectral method struggles with most (§5.2)
2. **Smaller training set:** 900 training samples here vs the 1000+ used in some FNO papers' own (differently-generated) Darcy experiments, combined with a genuinely smaller and more specific problem class (only 2 discrete permeability values vs a continuous field, which in principle should be *easier* to represent but

also means less variety to generalize from)

We explicitly avoid overclaiming a metric-definition artifact as the explanation here (a danger identified and fixed earlier in this project — see the note on `physical_rel_l2` in §3.4): this report's headline numbers are already computed in physical units, matching the literature convention, so the remaining gap reflects real differences in problem difficulty and dataset size, not a like-for-like metric mismatch.

6.3 Limitations

1. **Periodic assumption:** the FFT-based spectral convolution implicitly assumes periodic boundary conditions; the PDE itself uses Dirichlet (u≈0) boundaries — a known theoretical mismatch in vanilla FNO that padding/domain-extension tricks (not used here) can mitigate
2. **Fixed modes:** cannot adapt to varying frequency content across samples
3. **Latency vs parameter count:** as shown in §4.5, FNO's parameter efficiency does not automatically translate into lower wall-clock latency versus a dense-matmul baseline on modern GPU hardware, at single-sample inference
4. **2D, steady-state only:** no time dependence; 3D extension would increase compute significantly

---

7. Code Reproducibility

7.1 Repository Structure

```
ae646/
|-- configs/{fno,mlp,fno_improved}.yaml
|-- data/ # not tracked in git - regenerate via src/download_data.py + src/preprocess.py
|-- results/
| |-- run_001/ # FNO (original)
| |-- run_002/ # MLP baseline
| |-- run_003_fno_improved/
| `-- benchmark_speed.json
|-- src/
| |-- download_data.py, generate_data.py (optional fallback)
| |-- preprocess.py, models.py, train.py, evaluate.py
| |-- superres_eval.py, benchmark_speed.py, compare_comprehensive.py
|-- tests/
|-- requirements.txt / environment.yml
`-- README.md
```

7.2 Reproduction Commands

```
# Environment
conda env create -f environment.yml # or: pip install -r requirements.txt

# Data (real PDEBench, ~1.25 GB)
python src/download_data.py
python src/preprocess.py

# Train
```

```
python src/train.py --config configs/fno.yaml
python src/train.py --config configs/mlp.yaml
python src/train.py --config configs/fno_improved.yaml

# Evaluate
python src/evaluate.py --config configs/fno.yaml --checkpoint results/run_001/best_model.pt
python src/evaluate.py --config configs/mlp.yaml --checkpoint results/run_002/best_model.pt
python src/evaluate.py --config configs/fno_improved.yaml --checkpoint
results/run_003_fno_improved/best_model.pt

# Zero-shot super-resolution + inference-speed benchmark
python src/superres_eval.py --config configs/fno.yaml --checkpoint
results/run_001/best_model.pt
python src/superres_eval.py --config configs/fno_improved.yaml --checkpoint
results/run_003_fno_improved/best_model.pt
python src/benchmark_speed.py

# Comparison plots
python src/compare_comprehensive.py
```

7.3 Dependencies

See `requirements.txt` / `environment.yml`: torch, numpy, scipy, h5py, matplotlib, pyyaml, tqdm, wandb (optional, off by default), requests.

---

8. Conclusion

1. **FNO (4.7M params) achieves 0.0521 mean rel L2** vs **MLP (42.0M params) at 0.0820** — 36% lower error with 8.8× fewer parameters, on the real PDEBench dataset
2. **Improved FNO (78.8M params) reaches 0.0456** — a further 12.6% error reduction, but at 16.6× more parameters than the original FNO, which remains the more parameter-efficient choice
3. **Zero-shot super-resolution genuinely works:** both FNOs evaluate directly on real, native 128×128 PDEBench ground truth with only a moderate (14–24%) relative error increase, with no retraining
4. **Real speed measurement reveals a nuance the parameter counts alone don't show:** FNO and MLP are both ~1000×+ faster than a numerical FDM solve, but MLP is actually faster per-sample than FNO on this GPU — parameter efficiency and wall-clock latency are different things
5. The gap to commonly-cited FNO literature numbers (~0.01–0.02) is attributable to real, identified differences in dataset (PDEBench's piecewise-constant permeability is a harder input than the continuous field used in some other FNO Darcy experiments) and training set size — not to a metric-definition artifact, which was checked for and ruled out

---

9. Individual Contribution Statement

Aryaman Srivastava: All aspects — data source verification, preprocessing pipeline, model implementation (MLP, SpectralConv2d, FNO2d), training infrastructure, evaluation and visualization, zero-shot super-resolution and inference-speed benchmarking, report writing.

---

10. AI Tool Use Declaration

Tools used: Claude (Anthropic).

Usage: code implementation and debugging across the full pipeline (data download, preprocessing, model, training, evaluation, super-resolution, and benchmarking scripts); identifying and fixing a metric-computation bug and a broken data-source URL; locating and verifying the correct official PDEBench data source; report structuring and drafting.

Verification: all code was run and its output inspected by the student; results were independently cross-checked across two different machines/hardware configurations (Apple Silicon and an NVIDIA GPU), which agreed closely, as an additional reproducibility check. No AI-generated content was submitted without being read, understood, and verified against actual code output.

---

11. References

1. Li et al., "Fourier Neural Operator for Parametric PDEs", ICML 2021
2. Takamoto et al., "PDEBench: An Extensive Benchmark for Scientific Machine Learning", NeurIPS 2022
3. Kovachki et al., "Neural Operator: Learning Maps Between Function Spaces", 2023
4. PDEBench data: https://darus.uni-stuttgart.de/dataset.xhtml?persistentId=doi:10.18419/darus-2986
5. PDEBench repository: https://github.com/pdebench/PDEBench
6. NeuralOperator repository: https://github.com/neuraloperator/neuraloperator
7. Brunton & Kutz, "Data-Driven Science and Engineering", 2019

Appendix A: Hyperparameter Comparison (FNO)

Width	Modes	Layers	Params	Test Mean Rel L2
64	12	4	4.7M	**0.0521**
128	20	6	78.8M	0.0456

The original configuration is the more parameter-efficient operating point; the improved configuration trades a large increase in parameters for a moderate accuracy gain.

Appendix B: Complete Test Set Error Statistics

FNO (original): mean=0.0521, median=0.0398, std=0.0451, min=0.0155, max=0.3467 **MLP:** mean=0.0820, median=0.0695, std=0.0456, min=0.0325, max=0.3367 **FNO (improved):** mean=0.0456, median=0.0307, std=0.0487, min=0.0106, max=0.3535

(Full per-sample arrays: `results/run_*/evaluation/eval_metrics.json`)